

EXPERIMENT NO 9

Aim: Implement localStorage and sessionStorage to save user preferences like theme settings.

Requirements: A computer with a web browser, Visual Studio Code (or any text editor), HTML, CSS, and JavaScript.

Theory:

Modern web applications often need to remember information entered or selected by users. For example, a website may need to remember whether the user prefers Light Mode or Dark Mode, or temporarily remember a topic selected during the current browsing session.

JavaScript provides the Web Storage API for storing data directly in the user's browser. The two main types of browser storage are:

1. localStorage
2. sessionStorage

Both store data in the form of key-value pairs, where both the key and value are generally stored as strings.

1. localStorage

localStorage is used to store data that should remain available for a longer period.

The important characteristics of localStorage are:

- Data is stored in the user's browser.
- Data remains stored even after the browser is closed.
- Data remains available when the user revisits the website.
- It is associated with the website's origin.
- It can store data using key-value pairs.
- It does not automatically expire.
- It is suitable for storing persistent user preferences.

2. sessionStorage

sessionStorage is similar to localStorage, but it is designed for temporary storage.

Its characteristics are:

- Data is stored only for the current browser tab/session.
- Data is generally removed when the tab is closed.
- It is useful for temporary user selections.
- It also stores data as key-value pairs.
- It provides the same basic methods as localStorage.

Algorithm:

Step 1: Start.

Step 2: Create the required HTML elements such as a theme selector and topic selector.

Step 3: Add an event listener to the theme selector.

Step 4: When the user selects a theme, store the selected theme in localStorage.

Step 5: Retrieve the saved theme from localStorage when the page loads and apply it to the webpage.

Step 6: Add an event listener to the topic selector.

Step 7: Store the selected topic in sessionStorage.

Step 8: Retrieve the saved topic from sessionStorage when the page is loaded during the same session.

Step 9: Allow the user to clear the stored preferences using removeItem() or clear().

Step 10: Stop.

Conclusion:

Thus, the experiment successfully demonstrates the implementation of the Web Storage API in JavaScript. localStorage was used to maintain persistent user preferences, allowing settings such as the selected theme to remain available even after refreshing or reopening the webpage. sessionStorage was used to maintain temporary information such as the selected topic during the current browser session. The experiment shows that browser storage provides a simple and efficient way to maintain user preferences without requiring a server-side database. The use of appropriate storage mechanisms improves the usability and personalization of web applications while keeping temporary and permanent data separate.